

merci-audit:silence-style

Arquitectura Zero Maintenance: Compilación Incremental y st_mtime

Art de Côté | Cuadernillo

En mercedev.es, se enfrentaba un desafío crítico en el pipeline maestro (`merci-total.py`). Con el ecosistema documental creciente, el orquestador maestro estaba luchando por mantenerse eficiente. El tiempo de ejecución superaba los 20 segundos, lo que llevó a realizar una inyección del *Profiler* para identificar la causa.

El análisis reveló que el 90% del tiempo (aproximadamente 8.5 segundos) era consumido por el motor de Generación de Sitios Estáticos (SSG) al compilar PDFs mediante WeasyPrint. El sistema operaba bajo un patrón **Clean Build**: en cada ejecución, eliminaba a ciegas todos los archivos generados y los volvía a renderizar desde cero, independientemente de si el documento original (Markdown) había sido modificado o no.

Paralelamente, las herramientas de auditoría documental dependían de metadatos mantenidos manualmente por los humanos para calcular la deriva documental. Esto provocaba constantes falsos negativos debido a olvidos en la actualización de fechas escritas en comentarios de texto.

Para abordar estos problemas, se decidió implementar una refactorización transversal hacia el paradigma **Zero Maintenance** (Cero Mantenimiento Humano) e **Incremental Build** (Construcción Incremental):

1. **Adopción de la verdad física (`st_mtime`)**: Se erradicó la dependencia de las fechas manuales en los archivos de código. El detector de deriva y el motor SSG ahora interrogan directamente al Sistema Operativo para obtener la fecha inmutable de modificación física del archivo (`st_mtime`).
2. **Patrón Cache Hit**: El orquestador compara el `st_mtime` del PDF de salida frente al Markdown de origen. Si el origen no ha cambiado, se aborta la costosa llamada de renderizado (Cache Hit), reduciendo el tiempo de proceso de ese archivo a casi 0 milisegundos.
3. **Recolección de Basura (Mark & Sweep)**: Para evitar la acumulación de archivos "zombis" (PDFs/HTMLs cuyos Markdowns originales han sido borrados o renombrados), se implementó un *Garbage Collector* diferido. Durante la compilación, el script rastrea los archivos generados válidos en un `set()` . Al finalizar, itera sobre los directorios públicos y ejecuta un `unlink()` exclusivamente sobre los archivos huérfanos.
4. **Supply Chain Security**: Se endureció el Agente Auditor implementando el módulo nativo `ast` (Abstract Syntax Tree). En lugar de depender de expresiones regulares falibles, el auditor disecciona la

gramática de los scripts `.py` para bloquear cualquier importación que no figure en una lista blanca estricta (Zero Trust).

Migrar del *Clean Build* al *Incremental Build* redujo el tiempo de compilación del SSG de 8.46 segundos a **0.41 segundos** (una mejora del 95%), logrando que el pipeline maestro completo caiga por debajo de la barrera psicológica de los 10 segundos (Sub-10s).

Auditar la verdad física (`st_mtime`) y erradicar las cabeceras manuales demostró que el verdadero **DevSecOps** es aquel que elimina la carga cognitiva del desarrollador. Si una herramienta requiere que el humano introduzca datos redundantes para funcionar, la herramienta está mal diseñada.

Queda como conocimiento consolidado que la Inteligencia Artificial (SLMs), si bien es excepcional para razonar sobre código (como demostró el *Chaos Monkey*), añade una latencia inaceptable al ciclo crítico de Integración Continua (CI). Tareas de sincronización determinista, como la actualización del Roadmap o la recolección de métricas, deben delegarse incondicionalmente a código nativo (Python puro) para mantener la Experiencia de Desarrolladora (DX) intacta.

Resumen

A medida que el proyecto crecía, el sistema tardaba cada vez más en compilar porque borraba y volvía a construir todo desde cero en cada guardado. Se enseñó al orquestador a comprobar la fecha física del archivo en el disco duro para actualizar únicamente lo que realmente se ha tocado. Se ha pasado de esperar casi 10 segundos a que los cambios sean casi instantáneos, sin que ninguna persona tenga que anotar fechas a mano nunca más.

Para conocer la explicación detallada y los scripts involucrados, leer el [cuadernillo completo](#).